

Groundsill

Roadmap and Next Steps

Specification for the next development phase of the Git-backed information management platform

Field	Value
Document	Groundsill Roadmap and Next Steps
Status	Proposal for review
Version	1.0
Date	September 2026
Basis	The Origoa Foundation design guide and the implementation on the fable51-high branch
Audience	Collaborators planning and executing the next phase

Contents

1	Introduction.....	4
1.1	Purpose.....	4
1.2	Background.....	4
1.3	Design Goals for the Next Phase.....	4
1.4	Scope.....	4
1.5	Document Structure.....	5
2	Current State.....	6
2.1	Purpose.....	6
2.2	Architecture as Built.....	6
2.3	Delivered Capabilities.....	6
2.4	Verification Status.....	7
2.5	Known Limitations.....	7
2.6	Summary.....	7
3	Security Model.....	8
3.1	Purpose.....	8
3.2	Design Principles.....	8
3.3	Identity Providers.....	8
3.4	Sessions and Tokens.....	8
3.5	Commit Authorship.....	8
3.6	Authorization Model.....	9
3.7	Enforcement.....	9
3.8	Consequences.....	9
3.9	Implementation Plan.....	9
3.10	Design Summary.....	10
4	Operations.....	11
4.1	Purpose.....	11
4.2	Metrics.....	11
4.3	Structured Logging.....	11
4.4	Alert Guidance.....	11
4.5	Releases and Changelog.....	11
4.6	Implementation Plan.....	12
4.7	Design Summary.....	12
5	Schema Editing.....	13
5.1	Purpose.....	13
5.2	Impact Preview.....	13
5.3	Schema Editor.....	13
5.4	Workflow Editor.....	13
5.5	Implementation Plan.....	13
5.6	Design Summary.....	14
6	Scalability.....	15
6.1	Purpose.....	15
6.2	Batched Writes.....	15
6.3	Incremental Reindex.....	15
6.4	Attachments Outside Git.....	15
6.5	Presence Across Processes.....	16
6.6	Design Summary.....	16
7	Product Features.....	17
7.1	Purpose.....	17
7.2	Baselines.....	17
7.3	Import and Export.....	17
7.4	Bulk Operations.....	17
7.5	Rich Text.....	17

7.6 Saved Views and Notifications.....	18
7.7 Design Summary.....	18
8 Engineering Hygiene.....	19
8.1 Purpose.....	19
8.2 OpenAPI Description.....	19
8.3 Load and Soak Tests.....	19
8.4 Design Summary.....	19
9 Delivery Plan.....	20
9.1 Purpose.....	20
9.2 Order of Work.....	20
9.3 Milestones.....	20
9.3.1 Milestone A: Deployable.....	20
9.3.2 Milestone B: Ownable.....	20
9.3.3 Milestone C: Releasable.....	21
9.3.4 Milestone D: Scaled and Complete.....	21
9.4 Decisions Requested.....	21
9.5 Success Criteria.....	21
9.6 Design Summary.....	21
Appendix A: Effort Summary.....	22
Appendix B: API Additions.....	23

1 Introduction

1.1 Purpose

This document describes what Groundsill is today, what is still missing before it can be handed to a team, and how each missing piece should be built. It is written for a collaborator joining the project: every plan names the packages that exist in the code base so that work can start without a separate design phase, and every item carries an effort estimate so that the phase can be scheduled.

It complements the original design guide, which defines the architecture, the repository specification, the schema model and the minimum viable product. That guide has been implemented; this document is about the distance between the implemented product and a production deployment.

1.2 Background

Groundsill is a Git-backed storage platform for building information management applications such as requirements management, issue tracking, product lifecycle management and technical documentation. The domain model is configuration rather than code: schemas and workflows are JSON files in the repository, and a generic client renders whatever they define.

The implementation follows the design guide closely. A bare Git repository is the single source of truth. A PostgreSQL projection provides hierarchy queries, field indexes and full-text search, and can always be rebuilt from Git. Every write runs the repository update transaction of the guide: synchronize, validate, build an unpublished commit, project it, publish it with a compare-and-swap, and advance the processed revision. A Lit and TypeScript client talks only to the REST API and the WebSocket session service.

1.3 Design Goals for the Next Phase

- **Deployable to a team.** A verified identity for every action, roles per folder, metrics an operator can alert on, and a release process.
- **Owned by domain experts.** Schemas and workflows editable in the client, with a preview of what a change would break.
- **Scales with the repository.** Batched writes, a rebuild that no longer walks the full history, attachments outside the repository, presence across processes.
- **Fits the domain.** Baselines, import and export, bulk operations, rich text, saved views and notifications.
- **No new principles.** Every addition follows the existing ones: Git as truth, configuration over specialization, enforcement in the Foundation, a thin client.

1.4 Scope

In scope: every gap found during the production pass, ordered by how much it blocks a deployment, with a design and an implementation plan for each. Out of

scope: a rewrite of any existing layer, and features the design guide lists as not considered for the minimum viable product unless a customer has asked for them.

1.5 Document Structure

Chapter 2 records the current state so that the plans can be read against it. Chapters 3 to 8 cover the gaps in order of priority: security, operations, schema editing, scalability, product features and engineering hygiene. Chapter 9 turns the plans into a delivery order with milestones and the decisions the collaborator is asked to make. Appendix A summarizes effort and Appendix B lists the API additions.

2 Current State

2.1 Purpose

This chapter states what exists, how it was verified, and what is known to be missing, so that every later plan is an increment on a described baseline rather than an aspiration.

2.2 Architecture as Built

The server is a single Go binary, `groundsilld`, organized in layers that depend only downward:

Package	Responsibility
<code>cmd/groundsilld</code>	Flags and environment, HTTP server, security headers, optional access log, health probe wiring, graceful shutdown
<code>internal/httpapi</code>	REST layer and WebSocket hub: request decoding, ETag and If-Match, status codes, presence, same-origin sessions
<code>internal/foundation</code>	The service: every operation, the repository update transaction, service views, events
<code>internal/projection</code>	PostgreSQL projection: synchronization and replay, phased reindex, queries, validation, configuration cache
<code>internal/scanner</code>	Configurable repository scanner and the foundation indexer
<code>internal/gitx</code>	Bare-repository plumbing: compare-and-swap commits, batched reads, first-parent history walks
<code>internal/model</code>	Pure domain rules: identity, folders, fields, schema composition, workflows, overlays, content
<code>internal/ojson</code>	Order- and format-preserving JSON, so unchanged files are emitted byte for byte
<code>web/</code>	Lit and TypeScript client with a central store, URL router, REST client and session client

2.3 Delivered Capabilities

- Four artifact kinds (entries, documents, links, comments) with stable GUIDs, human-readable identifiers with history, overlays, folder metadata locality and structured commit messages.
- Lexical schema composition with nearest-wins resolution, every generic field type, link endpoint and cardinality rules, and workflows with per-schema assignment.
- A projection with 1tree hierarchy, key and value field index, full-text search, deleted-artifact tracking, GUID impostor detection, a four-phase

reindex with capabilities per phase, and maintenance mode shared across processes.

- A REST API for every artifact and service operation, a WebSocket session service with presence and commit and status events, and a health probe with the build version.
- A client with a + New flow that generates an empty form from the schema, a schema-driven detail view, a block-based document editor, relationships, comments, overlays, history, deep links, dark mode and a phone layout.
- Packaging: a container image, a compose file with PostgreSQL, a lint configuration, continuous integration with unit, race, fuzz, end-to-end and browser suites.

2.4 Verification Status

All layers are green on the current head: the Go suites under the race detector against isolated PostgreSQL schemas, the fuzz targets, the REST end-to-end script, and thirty-eight browser tests including a two-user collaboration scenario and adversarial cases. After every integration scenario the live projection is compared with a full rebuild, so the property that everything can be rebuilt from Git is verified on every run rather than assumed. The browser suite was also run three times in a row against one server with accumulating data, which is how the two defects fixed in the production pass were found.

2.5 Known Limitations

- No authentication or authorization; the commit author is self-reported.
- No metrics and no structured logging.
- Schemas and workflows are edited as JSON files, not in the client.
- Writes are serialized per process at roughly ten commits per second; a full rebuild walks the whole history; attachments live in Git; presence is per process.
- No baselines, import, export, bulk operations, rich text, saved views or notifications.
- No OpenAPI description, no load tests, no tagged releases.

2.6 Summary

The minimum viable product of the design guide is implemented, tested and packaged. What remains is the work that turns a correct system into one that a team can run and own, and it is listed in the following chapters in the order it should be done.

3 Security Model

3.1 Purpose

Today anyone who reaches the port can read and rewrite everything, and the commit author is whatever name the browser sends. This chapter defines identity, sessions, commit authorship and folder-scoped authorization. It is the first item of the next phase because nothing else can be deployed to a team without it.

3.2 Design Principles

- **Identity comes from an existing provider.** The server never becomes an identity provider and never stores passwords.
- **The verified identity reaches Git.** The commit author is the principal, never a client-supplied name, which turns the history into an audit trail.
- **Authorization is configuration.** Access rules are files in the repository, inherited like schemas and workflows.
- **Enforcement lives in the Foundation.** The HTTP layer and the WebSocket hub only carry the principal; every rule is checked once, in one place.

3.3 Identity Providers

The primary mode is built-in OpenID Connect using the authorization-code flow with PKCE against whatever the team already runs: Entra, Google, Keycloak or Okta. Discovery and token validation use `coreos/go-oidc`. The result is an encrypted, `HttpOnly`, `SameSite` session cookie, which keeps the single-binary deployment.

A `-auth proxy` mode trusts identity headers from a listed proxy address only, for teams that already run `oauth2-proxy` or `Authelia`. A `-auth none` mode keeps today's behaviour for local development and is never the default.

3.4 Sessions and Tokens

Session cookies are sealed with a key from `-session-key`, or a key generated on first start and persisted in the projection's `repo_state` row so that every process of one deployment shares it. Personal access tokens serve scripts, continuous integration and the seed script: generated in the client, stored as a hash in a new `tokens` table with subject, role, expiry and last use, and sent as a bearer header.

3.5 Commit Authorship

The Foundation builds the author name and email of every commit from the principal in the request context and rejects an author field in any request body. A `Groundsill-Actor` trailer records the subject so that audit queries over the history do not depend on display names. The presence hub uses the same verified name.

3.6 Authorization Model

An `access.json` file in a folder's `.groundsill` directory grants roles to users or identity-provider groups. Files are inherited down the tree with nearest-wins resolution, exactly like schemas and workflows, so membership is maintained once in the provider and the repository only records which groups may do what where. The root file is bootstrapped from an `-admin` flag on first start.

Role	May
reader	Read artifacts, attachments, relationships, comments and history in scope
editor	Create, update, move, transition, comment and attach in scope
maintainer	Edit schemas and workflows, move folders, delete artifacts in scope
admin	Reindex, relocate metadata, manage tokens and access files

3.7 Enforcement

A `Principal` type with subject, name, email and groups travels in the request context. Every Foundation operation resolves the effective role for the affected path through the projection, which indexes access files as a configuration category, so the check is one lookup. Reads are the subtle part: search and tree results must be filtered by permitted scopes, which maps directly onto the existing `ltree` hierarchy as one prefix filter on the allowed paths, resolved once per request.

3.8 Consequences

- Direct access to the bare repository is admin access, because a push bypasses the application. Only the service account may reach the server's repository, and mirrors that people push to need hooks on the hosting side. The installation guide states this.
- Cookies bring cross-site request forgery into scope. `SameSite=Lax` plus a required custom header on every state-changing request closes it; the client already sends JSON with a content type, so the change is small.

3.9 Implementation Plan

1. `internal/auth`: the `Principal` type, a context accessor, and the three authenticators behind one interface.
2. `internal/foundation`: operations take the principal from the context; `write()` builds the author from it and refuses a client-supplied author; the actor trailer.
3. `internal/httpapi`: authentication middleware, `GET /api/me`, `POST /api/logout`, the OIDC callback, the CSRF header check, token management routes and the tokens table.

4. `internal/model` and `internal/projection`: parse and compose `access.json`; index it; add the scope filter to Search and Tree.
5. `web/`: sign-in redirect on 401, the verified user in the header, a token page, and hiding of actions the role does not allow while the server keeps enforcing.
6. Tests: Foundation tests for every role boundary and the author override; HTTP tests for CSRF, the proxy allow-list and expired tokens; a browser test that a reader sees no edit controls and a direct write from that session is refused.

Order of work: proxy-header mode with verified authors and one all-or-nothing role first, about one day, which makes the system deployable behind what most teams already run; then built-in OpenID Connect and tokens; then folder-scoped roles with filtered reads.

Effort. *6 to 8 days in total.*

3.10 Design Summary

Identity from the provider, authorship from the identity, rules from the repository, enforcement in the Foundation. Nothing in the model is new to the system; it reuses the configuration resolution, the hierarchy index and the transaction that already exist.

4 Operations

4.1 Purpose

Logs and a health probe exist, but an operator cannot see the write queue, the projection lag or a rebuild's progress from outside, and there is no release process. This chapter adds metrics, structured logging, alert guidance, and tagged releases.

4.2 Metrics

Metrics use `prometheus/client_golang` and are served on a separate `-metrics-addr` listener, off by default, so they are never exposed on the application port.

Metric	Recorded in	Why it matters
<code>http_requests_total</code> , <code>http_request_duration_seconds</code> by route and status	the access-log middleware	latency and error rate per endpoint
<code>write_wait_seconds</code> , <code>write_attempts_total</code> by outcome, <code>commit_duration_seconds</code>	<code>foundation.write</code>	saturation of the serialized write path; stale retries and conflicts
<code>projection_lag_commits</code> , <code>reindex_duration_seconds</code> by phase, maintenance	projection	the projection falling behind; rebuild cost; time in maintenance
<code>ws_sessions</code> , database/sql pool statistics	the hub and the standard collector	live users; connection exhaustion

4.3 Structured Logging

`log/slog` replaces the package-level logger with one logger passed down. `-log-format text|json` selects the encoding. A request id is generated per request, echoed in an `X-Request-Id` response header and attached to every line the request produces, so one failing request can be followed through the layers.

4.4 Alert Guidance

The installation guide gains a section naming the metric to alert on and the threshold: write wait at the ninety-fifth percentile above one second, lag above zero for more than a minute, and maintenance longer than twice the last rebuild duration.

4.5 Releases and Changelog

Semantic version tags `vX.Y.Z`; the `-version` flag already reports the tag through linker flags. A release workflow on tag push builds binaries for `linux/amd64`, `linux/arm64` and `darwin/arm64` with `goreleaser`, attaches them and their checksums to the GitHub release, and pushes the container image to the GitHub

container registry. CHANGELOG.md follows the Keep a Changelog format and is maintained in every pull request; the workflow refuses a tag whose version has no changelog section. Before the first release that changes a table, a schema_version row in repo_state and forward-only migrations replace the current create-if-not-exists schema.

4.6 Implementation Plan

1. Register the metrics and the separate listener; instrument the four recording points.
2. Introduce the logger and request ids; convert existing log calls.
3. Write the alert section; add a metrics test that scrapes the registry after a request and a test that a failing request's log line carries its id.
4. Add the release workflow, the changelog and the version row.

Effort. *Observability 2 days; releases 1 day.*

4.7 Design Summary

Four groups of metrics on a private port, one logger with request ids, thresholds written down, and releases that are reproducible from a tag.

5 Schema Editing

5.1 Purpose

Schemas and workflows are JSON files managed through the API or through Git. That is fine for developers and a wall for everyone else. The design guide's promise is that the domain model is configuration owned by the domain; this chapter delivers the editor that makes the promise usable.

5.2 Impact Preview

Before any editor, the server learns to answer what a change would break. `POST /api/schemas/{type}/preview?scope=` takes a candidate schema and returns, without writing, the artifacts of that type in scope that would become invalid, each with the field and the reason, using the existing validation in `internal/model` against a candidate schema index. `POST /api/workflows/{id}/preview` does the same for artifacts whose current state would no longer exist. Both are cheap because the projection already indexes fields and states.

5.3 Schema Editor

A form over the schema document rather than a JSON text area: a field list with drag ordering; per-field type, required flag, enum options, reference target types; presentation columns; HID prefix and digits; workflow assignment. Link schemas get source and target types and cardinality. A JSON tab shows the exact file that will be committed, because the file is the contract. A scope chooser shows which folder's `.groundsill` directory the file lives in and which folders inherit it.

5.4 Workflow Editor

The existing state diagram becomes editable: states and transitions are added by clicking, and the initial state and transition names are edited in a side panel. The preview endpoint runs before saving.

5.5 Implementation Plan

1. The two preview endpoints in `httpapi` and `foundation`, with `Foundation` tests.
2. `web/src/schema-editor.ts` with the field list, the JSON tab and the scope chooser.
3. The editable workflow diagram.
4. The impact panel that runs the preview before saving and requires confirmation when something would break.
5. Browser tests: create a type from scratch, add an enum field, see the impact of removing a required field, edit a workflow transition.

Effort. 5 to 7 days.

5.6 Design Summary

The editor never invents a second model: it edits the same files the guide specifies, shows the file it will commit, and refuses to break existing artifacts silently.

6 Scalability

6.1 Purpose

Four limits will be felt as the repository grows: serialized writes, a rebuild that walks the whole history, binaries stored in Git, and presence that does not cross processes. None of them blocks a first deployment; all of them are planned so that they are addressed before they are felt.

6.2 Batched Writes

Commits are serialized per process at roughly ten per second, so a burst of creates from one client delays another client's single save by seconds. `POST /api/batch` accepts an ordered list of up to five hundred operations (create, update, delete, move, transition, link, comment) and executes them as one changeset in one commit through the existing transaction, which already builds multi-file changesets. The batch is all-or-nothing: the first validation failure aborts with the index of the failing operation. Operations may reference earlier ones by index, so a batch can create an entry and link it in one commit. The commit subject summarizes counts and one Groundsill-0p trailer per operation keeps the history greppable. The seed script, the end-to-end script and the browser fixtures switch to it, which also makes the suites faster.

Effort. *3 days.*

6.3 Incremental Reindex

The identity, field and full-text phases already build from the head tree and scale with the tree, not the history. Only the history phase scales with the number of commits. It becomes incremental: `repo_state` records a history cursor, the last commit whose events are stored; a rebuild keeps the history rows from before the cursor and walks only newer commits, unless the branch was rewound, which synchronization already detects, in which case it walks from the root as today. Capabilities are published per phase more finely, and writes are allowed once the field phase is complete, since the history phase never affects a write's validation. `POST /api/repository/reindex?history=defer` runs the first three phases now and the history phase in the background.

Effort. *3 days.*

6.4 Attachments Outside Git

A blob store behind an interface with two implementations, a directory and S3-compatible object storage, content-addressed by SHA-256. Attachments above a threshold, one megabyte by default, are stored there, and the repository holds a pointer file in the Git LFS pointer format, so standard `git lfs` clients read a mirror correctly and nothing custom is invented. The scanner classifies pointer files as attachments; downloads stream from the store with the existing ETag and sandbox headers; deletion removes the pointer and a `gc-blobs` command removes blobs that no tree in the history references.

Effort. *3 to 4 days.*

6.5 Presence Across Processes

PostgreSQL LISTEN and NOTIFY on a `groundsill_events` channel: every hub publishes its presence changes and the Foundation's commit and status events with a process id, and every hub subscribes on one dedicated, reconnecting connection and rebroadcasts to its own clients. Presence becomes a merge of every process's last state with a thirty-second heartbeat. The watch loop no longer needs to poll for other processes' commits; polling remains as the fallback for direct pushes.

Effort. *2 days.*

6.6 Design Summary

Each limit is removed by extending a mechanism that exists: the multi-file transaction, the rewind detection, the scanner's classification, and the database both processes already share.

7 Product Features

7.1 Purpose

These are the features a requirements or product lifecycle team asks for in the first weeks. They are ordered by how often they are asked for and by what they depend on.

7.2 Baselines

A baseline is a Git tag under `refs/tags/baselines/` on the managed branch, created through the plumbing with a compare-and-swap against a missing reference and an annotated message carrying the description and the actor. The API lists, creates and deletes baselines, and `GET /api/repository/diff?from=&to=&path=` uses the existing tree diff to list changed files, maps them to artifacts through the history table, and reports added, removed and changed artifacts with field-level differences computed on the two order-preserving JSON documents, so key order and formatting never appear as changes. `GET /api/artifacts/{guid}?at=` reads an artifact as of a baseline. The client gets a baselines list, a compare view and an as-of switch on the detail view.

Effort. *4 days.*

7.3 Import and Export

CSV import maps columns to fields by id or display name, with title, HID and folder as reserved columns; an existing HID means update. A dry run returns a row-by-row validation report using the same validation as single writes; a real run goes through the batch endpoint so one import is one commit. CSV export streams the current overview columns from the search endpoint. ReqIF import maps specification objects to entries, creating the schema when absent, relations to links and the hierarchy to a document that references the entries. Document export renders a document with resolved references as HTML with a print stylesheet, as PDF through headless Chromium when configured, and as DOCX through a pure-Go writer.

Effort. *CSV 2 days, ReqIF 5 days, export 3 days.*

7.4 Bulk Operations

A checkbox column with shift-click ranges and select-all in the overview, and a selection bar offering transition (only transitions valid for every selected artifact), move, set field and delete. Every bulk action is one batch request, so it is one commit with per-item outcomes, and the bar lists failures with links.

Effort. *2 days, after batched writes.*

7.5 Rich Text

Inline marks are stored as a constrained Markdown subset inside block text: bold, italic, code and links. Storage stays text, so full-text extraction, diffs and size limits are untouched and no HTML is ever stored. A small parser renders marks

to DOM nodes directly, never through HTML injection, and the server validates link schemes as it already does for fields. A table block with rows of cells gets a grid editor. The editor adds a floating toolbar on selection, keyboard shortcuts, and paste handling that strips formatting to the subset.

Effort. *4 days.*

7.6 Saved Views and Notifications

Shared saved views are configuration files under `.groundsill/views/`, inherited like schemas and listed in the sidebar; personal views live in a preferences table because they are not repository content. Subscriptions to an artifact or a folder are stored per user, and a dispatcher matches the Foundation's existing commit events to them. Channels are webhooks with an HMAC signature, retries and a dead-letter list, and email through SMTP with a daily digest. The client gets a watch toggle, a notifications page and view management.

Effort. *4 days, after the security model.*

7.7 Design Summary

Baselines are tags, views are configuration, imports are batches, rich text is text. Each feature is expressed in the repository's own vocabulary so that it stays rebuildable, diffable and readable with plain Git.

8 Engineering Hygiene

8.1 Purpose

Two items that cost little and prevent expensive surprises once external users appear.

8.2 OpenAPI Description

`api/openapi.yaml` is written by hand; the routes are stable and few enough that a generator would add noise. It is served at `GET /api/openapi.json`. A test walks every registered route and fails when the description does not mention it, and vice versa, so the two cannot drift. The client's types are generated from the description in the web build, replacing the hand-written ones, and a rendered version is served offline so the content security policy stays strict.

Effort. *2 days.*

8.3 Load and Soak Tests

k6 scripts under `perf/`: a create burst, a mixed profile modelled on the browser suite's request pattern, a search-heavy profile, and a reindex during load. A nightly job runs them against the compose deployment seeded with ten thousand artifacts, with thresholds that fail the job: ninety-fifth percentile reads under one hundred milliseconds, single writes under three hundred milliseconds at twenty virtual users, no server errors. A weekly one-hour soak samples the metrics endpoint for goroutine, connection and memory growth. The numbers are published in the installation guide under a capacity heading.

Effort. *2 days, after metrics.*

8.4 Design Summary

The description and the tests are both enforced by the build, so they describe the system as it is rather than as it was.

9 Delivery Plan

9.1 Purpose

This chapter orders the work, groups it into milestones with a clear definition of done, and lists the decisions that the collaborator is asked to make before the first milestone starts.

9.2 Order of Work

Step	Items	Why now	Effort
1	Security: proxy mode, verified authors, one role	Deployable behind what teams already run	1 day
2	Operations: metrics, logging, alerts	Needed before any real traffic	2 days
3	Security: OpenID Connect, tokens, folder roles	Complete the security model	5 to 7 days
4	Batched writes, then bulk operations	One endpoint unlocks batch, import and bulk	5 days
5	Schema and workflow editor	Lets non-developers own the domain model	5 to 7 days
6	Releases, OpenAPI, load tests	Hygiene before the first external users	5 days
7	Baselines, CSV import and export	The two most requested features in this domain	6 days
8	Incremental reindex, attachments, presence	Scale work, when repository size demands it	8 to 9 days
9	Rich text, views and notifications, ReqIF and document export	Breadth	16 days

9.3 Milestones

9.3.1 Milestone A: Deployable

Steps 1 and 2. Done when a team can run the container behind their proxy, every commit carries a verified author, and an operator has a dashboard and two alerts. About three days.

9.3.2 Milestone B: Ownable

Steps 3 to 5. Done when folder roles are enforced and tested, a bulk transition is one commit, and a domain expert has created a type and a workflow without touching JSON. About three weeks.

9.3.3 Milestone C: Releasable

Steps 6 and 7. Done when version 1.0.0 is tagged with binaries and an image, the API is described and generated types are in use, the nightly load job is green, and a baseline can be created and compared. About two weeks.

9.3.4 Milestone D: Scaled and Complete

Steps 8 and 9, scheduled by demand rather than by calendar.

9.4 Decisions Requested

- **Identity provider.** Which provider the first deployment uses, and whether the proxy mode is enough for it. This decides whether step 3 can follow step 1 immediately.
- **Group claims.** Whether the provider exposes group membership in the token, which decides whether access files reference groups or individual users.
- **Object storage.** Whether an S3-compatible store is available, or attachments stay on a directory volume.
- **Export formats.** Whether DOCX is required or HTML and PDF are enough for the first release.
- **Ordering.** Whether baselines should move ahead of the schema editor for the first customer.

9.5 Success Criteria

- Every write in the history is attributable to a verified identity.
- A reader role cannot obtain, through any endpoint, data outside its scope; the test suite proves it.
- The load job stays within its thresholds at ten thousand artifacts and twenty concurrent users.
- A domain expert creates a schema, a workflow and a baseline without a developer.
- All existing suites stay green and the live projection still matches a full rebuild after every scenario.

9.6 Design Summary

Security first, then visibility, then ownership by the domain, then release hygiene, then breadth and scale as demand appears. Each milestone leaves the system in a state that is better than the previous one and never in a half-migrated state.

Appendix A: Effort Summary

Item	Effort (days)	Depends on
Security model	6 to 8	none
Operations: observability	2	none
Operations: releases	1	none
Schema and workflow editor	5 to 7	none
Batched writes	3	none
Incremental reindex	3	none
Attachments outside Git	3 to 4	object storage decision
Presence across processes	2	none
Baselines	4	none
CSV import and export	2	batched writes
ReqIF import	5	batched writes, CSV
Document export	3	export format decision
Bulk operations	2	batched writes
Rich text	4	none
Saved views and notifications	4	security model
OpenAPI description	2	none
Load and soak tests	2	observability
Total	53 to 60	

Appendix B: API Additions

Endpoint	Chapter	Purpose
GET /api/me, POST /api/logout, OIDC callback, token routes	3	identity, sessions, personal access tokens
GET /metrics on the metrics listener	4	Prometheus metrics
POST /api/schemas/{type}/preview, POST /api/workflows/{id}/preview	5	impact of a configuration change before saving
POST /api/batch	6	many operations in one commit
POST /api/repository/reindex?history=defer	6	rebuild with a deferred history phase
GET, POST, DELETE /api/repository/baselines	7	named, tagged revisions
GET /api/repository/diff?from=&to=&path=	7	artifact and field differences between revisions
GET /api/artifacts/{guid}?at=	7	an artifact as of a baseline
POST /api/import/csv, POST /api/import/reqif	7	imports with dry runs
GET /api/repository/search?format=csv, GET /api/documents/{guid}/export	7	exports
views, subscriptions and notification routes	7	saved views, watching, delivery channels
GET /api/openapi.json, GET /api/docs	8	the API description